

A.ai

Free Hosting Deployment Guide

Step by step: vercel.app + onrender.com, no custom domain, one GitHub repository

Repository: github.com/Ajay-Dabhi-99/A.AI

Stack: Vercel Hobby (website + API forwarding) + Render Free (API) + Supabase + Upstash + Resend

Version 1.0 · 17 September 2026 · Author: Dabhi Ajay

Contents

1. Read this first.....	3
1.1 The one problem with free addresses, and the fix	3
1.2 What changes compared with the custom-domain setup	3
1.3 Limits of the free setup (know these before you share the link)	4
2. How your single GitHub repository runs everything.....	5
2.1 Where each setting is kept	5
2.2 The branch flow	5
3. Step-by-step setup	6
Phase A: your computer and repository	6
Phase B: Supabase (database and storage)	6
Phase C: Upstash (Redis)	7
Phase D: email (Resend)	7
Phase E: AI keys and the login secret.....	7
Phase F: Render (API) on the free plan	8
Phase G: Vercel (website) with API forwarding	9
Phase H: test everything	11
Phase I: GitHub (protect main and schedule cleanup)	11
Phase J: after going live	12
4. Everyday work: releasing a change	13
4.1 Database changes.....	13
4.2 Changing a setting.....	13
5. Optional: a gated release button instead of auto-deploy	14
6. Rollback and backups	15
6.1 Undo a bad release	15
6.2 Backups	15
7. Troubleshooting.....	16
Appendix A: your final vercel.json	17
Appendix B: settings summary.....	18
Render (a-ai-api, Free)	18
Vercel (website)	18
GitHub	18
Appendix C: checklist.....	19

1. Read this first

This guide is for the setup you already have: the website on a free **Vercel** address (**YOUR-APP.vercel.app**) and the API on a free **Render** address (**YOUR-API.onrender.com**), with **no domain of your own**. It explains, step by step, how to make that setup work completely and how to manage it from your single GitHub repository.

Throughout the guide replace:

Placeholder	Replace with	Where to find it
YOUR-APP.vercel.app	Your Vercel production address	Vercel → your project → Settings → Domains (the one ending in .vercel.app without random letters)
YOUR-API.onrender.com	Your Render service address	Render → your web service → the URL at the top of the page

1.1 The one problem with free addresses, and the fix

A.ai keeps people signed in with secure cookies (**__Host-a_ai_session**, **__Host-a_ai_guest**, marked **SameSite=Lax**). A browser only sends such cookies to the **same site** that the page is on.

- **YOUR-APP.vercel.app** and **YOUR-API.onrender.com** are **two different sites**.
- If the website calls the Render address directly, the browser drops the cookies. Result: sign-in "succeeds" but you are immediately signed out, guest chat fails, and history is empty.

The fix (no app code change, one config file): let Vercel forward API calls to Render. The browser only ever talks to **YOUR-APP.vercel.app**; Vercel passes **/api/...** requests on to Render behind the scenes. For the browser, the website and the API are then the same site, so cookies work.

```

Browser
  | https://YOUR-APP.vercel.app/          -> website files (Vercel)
  | https://YOUR-APP.vercel.app/api/...   -> Vercel forwards to Render
  v
Vercel  --(rewrite)--> https://YOUR-API.onrender.com/api/... (Fastify API)
                        |
                        +-- Supabase (PostgreSQL + Storage)  Upstash (Redis)
                        +-- AI providers  Resend (email)

```

This is the same trick the project already uses on your computer: in development the Vite server forwards **/api**, **/health** and **/ready** to the API (**apps/web/vite.config.ts**), and **VITE_API_URL** is left empty.

1.2 What changes compared with the custom-domain setup

Setting	Custom domain setup	Your free setup
Website address	app.yourdomain.com	YOUR-APP.vercel.app
Address the browser uses for the API	api.yourdomain.com	YOUR-APP.vercel.app/api (forwarded)
VITE_API_URL (Vercel)	https://api.yourdomain.com	empty / deleted
CORS_ORIGIN , APP_URL (Render)	https://app.yourdomain.com	https://YOUR-APP.vercel.app
TRUST_PROXY_HOPS (Render)	1	2 (Vercel + Render)
apps/web/vercel.json	as in the repo	add API rewrites (Step 17)
Database migrations	Render pre-deploy command	Build command (free plan has no pre-deploy)
Daily cleanup job	Render cron job	GitHub Actions schedule (free)
Admin promotion	Render Shell	From your computer (free plan has no Shell)

1.3 Limits of the free setup (know these before you share the link)

Limit	What it means for A.ai	What you can do
Render free instances sleep after 15 minutes without traffic; waking takes about 1 minute	The first visitor after a quiet period waits ~1 minute before the chat answers.	Accept it, or ping <code>/health</code> every 10 minutes (Step 27), or upgrade to Starter later.
Render free: 750 instance hours per month per workspace	One service running all month uses about 744 hours. A second free service would run out.	Keep only one free web service.
Render free: no pre-deploy command, no Shell, no cron jobs	Migrations, admin promotion and cleanup need another way.	Steps 11, 24 and 25 of this guide.
Vercel proxied requests time out after 120 seconds	An answer that takes longer than 2 minutes to start can fail through the proxy.	Normal chats are far shorter. If very long replies fail, see Section 7.
Resend without a verified domain only delivers to your own Resend email	Other people cannot receive the verification email, and A.ai does not allow sign-in until the email is verified . Guests can still chat.	Buy any cheap domain just for email and verify it in Resend (Step 7). The website can stay on vercel.app.
Supabase free projects pause after a period of inactivity	The API cannot reach the database and <code>/ready</code> fails.	Open the Supabase dashboard and click Restore ; use the app regularly.
Free AI provider keys have small quotas	Busy days can hit provider limits; A.ai falls back to another model when it can.	Set more than one provider key.

2. How your single GitHub repository runs everything

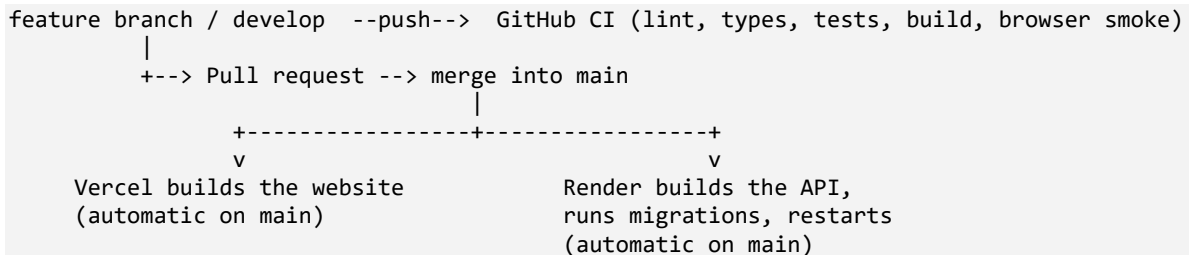
Your repository `github.com/Ajay-Dabhi-99/A.AI` is a **monorepo**: the website (`apps/web`), the API (`apps/api`), shared packages (`packages/*`) and the database migrations (`prisma/migrations`) all live in it. You do not need a second repository. Each service simply reads a different folder of the same repo.

Service	Connected to	Reads from the repo	Produces
Vercel	Your repo, branch <code>main</code>	Root Directory <code>apps/web</code> + <code>apps/web/vercel.json</code>	The website, plus the <code>/api</code> forwarding rule
Render	Your repo, branch <code>main</code>	Build and start commands (dashboard or <code>render.yaml</code>)	The API; applies migrations during the build
GitHub Actions	Your repo	<code>.github/workflows/*.yaml</code>	Tests on every push/PR; daily cleanup; optional Deploy button
Supabase, Upstash, Resend	Not connected to Git	—	Receive connections from the API

2.1 Where each setting is kept

Setting	Kept in	In Git?
Build rules, forwarding rules, security headers	<code>apps/web/vercel.json</code>	Yes
API secrets (database, Redis, JWT, keys)	Render → service → Environment	Never
Cleanup job secrets	GitHub → Settings → Secrets and variables → Actions	Never
Your local development values	<code>.env</code> in the repo root (git-ignored)	Never

2.2 The branch flow



This guide uses **automatic deploys on `main`** because it is the simplest reliable flow for free plans. Protect `main` (Step 22) so only tested code reaches it. A manual, gated release is optional (Section 5).

3. Step-by-step setup

If you already did some of these steps, still read each one: several settings must change for the free setup (they are marked **CHANGE**).

Phase A: your computer and repository

Step 1 Check the tools

1. `node -v` shows **v22.13** or newer (Render uses 22.16.0).
2. `corepack enable`, then `pnpm -v` shows **10.x**.
3. In the project folder run `pnpm install`, then `pnpm verify`. Everything must pass.

Step 2 Make sure GitHub has your latest code

1. Commit your work and push to **main** (or merge a pull request).
2. Open **GitHub** → **A.AI** → **Actions**. The latest **CI** run must be green.

NOTE

Vercel and Render build what is on GitHub. Uncommitted changes on your computer are never deployed.

Phase B: Supabase (database and storage)

Step 3 Create or open the production project

1. **supabase.com** → **New project** (or open the one you already use). Name it for example **a-ai-production**.
2. Region: close to your Render region (for Render Singapore choose **Southeast Asia (Singapore)**).
3. Save the database password in a password manager.

Step 4 Copy the two connection strings (CHANGE: use the poolers)

1. Click **Connect** at the top of the project.
2. **Transaction pooler** (host `...pooler.supabase.com`, port **6543**) → this is `DATABASE_URL`.
3. **Session pooler** (same host, port **5432**) → this is `DIRECT_URL`.
4. Replace `[YOUR-PASSWORD]` in both. If the password has symbols like `@ # / ?`, URL-encode them (`@` → `%40`) or pick a letters-and-digits password.

IMPORTANT

Do not use the "Direct connection" string (`db.xxxx.supabase.co:5432`) on Render. It is IPv6-only on Supabase, and hosts without IPv6 cannot reach it, so migrations fail. The **Session pooler** string works over IPv4 and supports migrations.

Step 5 Create the private storage bucket (for image uploads)

1. **Storage** → **New bucket**, name exactly **a-ai-attachments**.
2. **Public bucket: OFF**. Create.
3. **Project Settings** → **API**: copy **Project URL** → `SUPABASE_URL`, and the **service_role** key → `SUPABASE_SERVICE_ROLE_KEY`.

IMPORTANT

The `service_role` key is a master key. Put it only in Render and in GitHub Actions secrets. Never in Vercel, never in Git.

Phase C: Upstash (Redis)

Step 6 Create the Redis database

1. **console.upstash.com** → **Redis** → **Create Database**. Name **a-ai-production**, region near Render, **TLS on**.
2. Open it → **Connect** → **ioredis / TCP**. Copy the URL that begins with **rediss://** → **REDIS_URL**.

NOTE

It must start with **rediss://** (with TLS). In production the API refuses to start with **redis://**.

Phase D: email (Resend)

Step 7 Choose how email will work

The production API **will not start** without **RESEND_API_KEY**, and users **cannot sign in until they verify their email**. Pick one option:

Option	Who can sign up	What to do
A. Test mode (no domain)	Only you (the email you used to create the Resend account). Everyone else can use guest chat only.	Create an API key. Keep EMAIL_FROM=A.ai <onboarding@resend.dev> .
B. Real users (recommended)	Everyone	Buy any cheap domain (only used for email), add it in Resend → Domains , add the DNS records Resend shows, wait for Verified . Set EMAIL_FROM=A.ai <no-reply@your-email-domain.com> .

1. **resend.com** → **API Keys** → **Create API Key** (Sending access). Copy it → **RESEND_API_KEY**.
2. Decide **EMAIL_FROM** from the table above.

TIP

Email links point to **APP_URL**, which will be **https://YOUR-APP.vercel.app**. A domain used only for Resend does not change your website address.

Phase E: AI keys and the login secret

Step 8 Collect at least one AI provider key

Provider	Where	Variable
Google Gemini	aistudio.google.com → Get API key	GEMINI_API_KEY
Groq (also turns on voice input)	console.groq.com → API Keys	GROQ_API_KEY
OpenRouter	openrouter.ai → Keys	OPENROUTER_API_KEY

With **no** key, **/ready** reports not ready. Setting two or three keys gives A.ai a fallback when one provider is busy.

Step 9 Generate JWT_SECRET

```
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
```

Copy the output → **JWT_SECRET** (at least 32 characters). Changing it later signs everybody out.

Phase F: Render (API) on the free plan

Step 10 Create (or check) the web service

The simplest way on the free plan is a normal web service, set up in the dashboard. (The repo's **render.yaml** Blueprint is written for the paid Starter plan: it uses a pre-deploy command and a cron job, which the free plan does not support.)

1. **dashboard.render.com** → **New +** → **Web Service**. (If you already have the service, open it and go to **Settings** instead.)
2. Connect GitHub and pick **Ajay-Dabhi-99/A.AI**.
3. Fill in the form exactly:
 - **Name:** **a-ai-api** (this becomes **a-ai-api.onrender.com** or similar)
 - **Region:** Singapore (or the region nearest your Supabase)
 - **Branch:** **main**
 - **Root Directory:** leave **empty** (the build needs the whole repo)
 - **Runtime / Language:** Node
 - **Instance Type:** Free

Step 11 Set the build and start commands (CHANGE: migrations in the build)

Build Command (one line):

```
corepack enable && NODE_ENV=development pnpm install --frozen-lockfile && pnpm turbo run build --filter=@a-ai/api... && pnpm db:deploy
```

Start Command:

```
node apps/api/dist/server.js
```

- **NODE_ENV=development** during install is on purpose: otherwise pnpm skips TypeScript and turbo, and the build fails. The running app still uses **NODE_ENV=production**.
- **pnpm db:deploy** at the end applies any new migrations from **prisma/migrations** on every deploy. It is safe to run repeatedly; already-applied migrations are skipped.

Then in **Settings: Health Check Path** = **/health**. **Auto-Deploy** = **On Commit** (Yes).

Step 12 Add the environment variables

Render → **a-ai-api** → **Environment** → **Add Environment Variable**. Add every row:

Key	Value
NODE_VERSION	22.16.0
NODE_ENV	production
HOST	0.0.0.0
LOG_LEVEL	info
TRUST_PROXY_HOPS	2 (CHANGE: Vercel + Render are two proxies)
DATABASE_URL	Transaction pooler string, port 6543 (Step 4)
DIRECT_URL	Session pooler string, port 5432 (Step 4)
REDIS_URL	rediss://... (Step 6)
JWT_SECRET	From Step 9

Key	Value
CORS_ORIGIN	<code>https://YOUR-APP.vercel.app</code> (CHANGE: exact, no slash at the end)
APP_URL	<code>https://YOUR-APP.vercel.app</code> (CHANGE)
RESEND_API_KEY	From Step 7
EMAIL_FROM	From Step 7
SUPABASE_URL	From Step 5
SUPABASE_SERVICE_ROLE_KEY	From Step 5
SUPABASE_STORAGE_BUCKET	<code>a-ai-attachments</code>
GEMINI_API_KEY / GROQ_API_KEY / OPENROUTER_API_KEY	The keys you have (Step 8). Do not add empty ones.
SENTRY_DSN	Optional. Leave out to disable.

Click **Save, rebuild, and deploy** (or **Save Changes** and then **Manual Deploy** → **Deploy latest commit**).

NOTE

Why `TRUST_PROXY_HOPS=2`: A.ai applies per-visitor limits using the visitor's IP address (300 API requests per minute, login attempts, the guest daily quota). Behind Vercel **and** Render there are two proxies. With the default `1`, the API would see Vercel's address for everyone, and all your visitors would share one limit.

Trade-off: someone who calls `YOUR-API.onrender.com` directly can fake their address to dodge per-IP limits. Per-user and per-guest limits still apply.

Step 13 Watch the first deploy

1. Open `a-ai-api` → **Logs** (or **Events** → the running deploy).
2. You should see: `install` → `build` → `prisma migrate deploy` ("All migrations have been successfully applied" or "No pending migrations") → `Your service is live`.
3. If it stops with **Invalid environment configuration**, the log lists the wrong variable names (never values). See Section 7.

Step 14 Test the API directly

```
curl https://YOUR-API.onrender.com/health
curl https://YOUR-API.onrender.com/ready
```

Expected: `/health` → `{"status":"ok",...}`. `/ready` → HTTP 200 and `"status":"ready"`. If the service was asleep, the first call can take about a minute.

Phase G: Vercel (website) with API forwarding

Step 15 Create (or check) the Vercel project

1. `vercel.com` → **Add New...** → **Project** → **Import** `Ajay-Dabhi-99/A.AI`. (If it exists, open **Settings** → **Build and Deployment**.)
2. **Root Directory:** `apps/web`. This is the key setting for the single repo.
3. Framework, Install, Build and Output commands: leave as detected. `apps/web/vercel.json` sets them (install and build run from the repo root).
4. **Production Branch** (Settings → Environments → Production, or Git): `main`.

Step 16 Remove VITE_API_URL (CHANGE)

1. **Vercel** → **project** → **Settings** → **Environment Variables**.
2. If **VITE_API_URL** exists, **delete it** (or set it to an empty value). The website must call its own address (**/api/...**), not the Render address.
3. Optional: **VITE_SENTRY_DSN** for browser error reports.

IMPORTANT

VITE_ values are built into the website. After changing them you must redeploy (Step 18).

Step 17 Add the API forwarding rules to vercel.json (CHANGE, one-time config edit)

Open **apps/web/vercel.json**. Replace the **"rewrites"** line with the block below, using **your** Render address. The API rules must come **before** the last rule (which sends every other path to the React app):

```
"rewrites": [
  { "source": "/api/:path*", "destination": "https://YOUR-API.onrender.com/api/:path*" },
  { "source": "/health", "destination": "https://YOUR-API.onrender.com/health" },
  { "source": "/ready", "destination": "https://YOUR-API.onrender.com/ready" },
  { "source": "/(.*)", "destination": "/index.html" }
],
```

Then, inside the existing **"headers": [...]** list, add this entry so Vercel never caches API answers:

```
{
  "source": "/api/(.*)",
  "headers": [{ "key": "x-vercel-enable-rewrite-caching", "value": "0" }]
},
```

Check and publish the change:

```
pnpm format
pnpm verify
git add apps/web/vercel.json
git commit -m "chore(web): forward /api to the Render API on free hosting"
git push origin main
```

NOTE

The website code needs no change: when **VITE_API_URL** is empty it calls relative paths like **/api/chat** (**apps/web/src/services/api.ts**), which now go through Vercel to Render.

Step 18 Redeploy and check the website

1. The push in Step 17 starts a Vercel deployment automatically. Wait until **Deployments** shows **Ready**.
2. If you only changed environment variables, use **Deployments** → ... → **Redeploy**.
3. Open **https://YOUR-APP.vercel.app/health** in the browser. You should see the API's **{"status":"ok",...}** answer. That proves forwarding works.
4. Open **https://YOUR-APP.vercel.app**. The A.ai home page loads.

Step 19 Check the browser really uses the forwarding

1. Open the site, press **F12** → **Network**, and send a guest chat message.
2. The requests must go to **YOUR-APP.vercel.app/api/...**, **not** to **onrender.com**.
3. In **Application** → **Cookies** → **YOUR-APP.vercel.app** you should see **__Host-a_ai_guest** (and after sign-in **__Host-a_ai_session**).

If requests still go to onrender.com, **VITE_API_URL** is still set somewhere: fix Step 16 and redeploy.

Phase H: test everything

Step 20 Run the release smoke test from your computer

```
pnpm smoke --api https://YOUR-APP.vercel.app --web https://YOUR-APP.vercel.app
```

Using the Vercel address for both tests the whole path (browser → Vercel → Render). Each line prints PASS or FAIL: health, readiness, security headers, error format, CORS, cookie flags, model list, website security policy and deep links. It calls **no** AI provider. You can also test Render alone:

```
pnpm smoke --api https://YOUR-API.onrender.com --web https://YOUR-APP.vercel.app
```

Step 21 Test the site by hand

1. Open the site in a private window. Chat as a guest (up to 20 messages per day by default).
2. Sign up with first and last name. Open the verification email and click the link. (Test mode: only your own Resend email receives it.)
3. Sign in, chat, reload the page: the conversation stays.
4. Compare two models. Try an image upload (needs Supabase Storage) and the microphone (needs `GROQ_API_KEY`).
5. Open history and the dashboard. Sign out and sign in again.

Phase I: GitHub (protect main and schedule cleanup)

Step 22 Protect the main branch

1. **GitHub** → **A.AI** → **Settings** → **Branches** → **Add branch ruleset** (target `main`).
2. Turn on **Require a pull request before merging**.
3. Turn on **Require status checks to pass** and add `Typecheck`, `lint`, `test`, `build` and `Browser smoke` (`Playwright`, `mocked API`).
4. Save. Because Vercel and Render deploy every commit on `main`, this is what keeps broken code off the live site.

Step 23 Add the secrets for the daily cleanup job

Uploaded images that were never sent are removed by `pnpm attachments:cleanup`. Render free has no cron jobs, so GitHub Actions will run it (free for public repos).

1. **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.
2. Add `DATABASE_URL`, `SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY` (same values as Render).

Step 24 Add the cleanup workflow file

Create `.github/workflows/cleanup.yml` with this content, then commit and push it (through a pull request):

```
# Daily removal of image uploads never sent with a message (ADR-015).
name: Attachments cleanup

on:
  schedule:
    - cron: '30 3 * * *'
  workflow_dispatch:

permissions:
  contents: read

jobs:
  cleanup:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
```

```

- uses: actions/checkout@v4
- uses: pnpm/action-setup@v4
- uses: actions/setup-node@v4
  with:
    node-version: 22
    cache: pnpm
- run: pnpm install --frozen-lockfile
- run: pnpm attachments:cleanup
env:
  DATABASE_URL: ${ secrets.DATABASE_URL }
  SUPABASE_URL: ${ secrets.SUPABASE_URL }
  SUPABASE_SERVICE_ROLE_KEY: ${ secrets.SUPABASE_SERVICE_ROLE_KEY }
  SUPABASE_STORAGE_BUCKET: a-ai-attachments

```

Test it once: **Actions** → **Attachments cleanup** → **Run workflow**. It must finish green.

Phase J: after going live

Step 25 Make yourself admin (from your computer)

1. Sign up on the live site and verify your email.
2. On your computer, make sure the root `.env` has the **production** `DATABASE_URL` (the same one as Render).
3. Run:

```
pnpm admin:promote your-email@example.com
```

To remove admin rights later: add `--revoke`. There is deliberately no button or API for this.

Step 26 Model list (nothing to do)

The API adds the default models to the registry the first time it needs them. To do it yourself: `pnpm db:seed` from your computer (it never overwrites admin changes).

Step 27 Stop Render from sleeping (optional)

1. Create a free monitor at a service such as **UptimeRobot** or **cron-job.org**.
2. URL: `https://YOUR-API.onrender.com/health`, interval: **10 minutes**, method GET.
3. The service then stays awake. One free service running all month uses about 744 of the 750 free hours, so do not keep a second free service running.

NOTE

Ping `/health`, not `/ready`: `/health` does not touch the database or Redis, so it does not use your Upstash command quota.

4. Everyday work: releasing a change

1. Create a branch: `git checkout -b feature/my-change`.
2. Change the code. Run `pnpm verify`.
3. Push the branch and open a pull request into `main`. Wait for CI to go green.
4. Merge. Vercel and Render now deploy automatically (Render also runs any new migrations in its build).
5. Wait until **Vercel** → **Deployments** shows Ready and **Render** → **Events** shows "Deploy live".
6. Run `pnpm smoke --api https://YOUR-APP.vercel.app --web https://YOUR-APP.vercel.app` and try the site quickly.

4.1 Database changes

- Create the migration locally with `pnpm db:migrate`, commit the new folder in `prisma/migrations`.
- Only add things (tables, nullable columns, columns with defaults, indexes). Remove or rename columns only in a later release.
- Never edit a migration that is already deployed; add a new one.
- Every new table must enable Row Level Security in the same migration.
- Run `pnpm test:live` when database or Redis behaviour changes.

4.2 Changing a setting

Setting	Where	Takes effect
API secret or limit	Render → Environment → Save	After Render restarts (automatic)
Website variable (<code>VITE_*</code>)	Vercel → Environment Variables	Only after Redeploy
Forwarding rules	<code>apps/web/vercel.json</code> (via pull request)	On the next Vercel deploy
Cleanup secrets	GitHub → Secrets and variables → Actions	Next scheduled run
Render address changed	Update the 3 rewrites in <code>vercel.json</code>	On the next Vercel deploy
Vercel address changed	Update <code>CORS_ORIGIN</code> and <code>APP_URL</code> on Render	After Render restarts

5. Optional: a gated release button instead of auto-deploy

The repository also contains `.github/workflows/deploy.yml`. It re-runs every test (including live database tests), then triggers Render and Vercel, waits for the new version and runs the smoke test. Use it when you want releases only after that full check.

1. Create a **separate test** Supabase project and Upstash database (never production; the tests write data).
2. GitHub → **Settings** → **Environments** → **New environment** `test`: secrets `TEST_DATABASE_URL` (transaction pooler), `TEST_DIRECT_URL` (session pooler), `TEST_REDIS_URL`.
3. Environment `production`: secrets `RENDER_DEPLOY_HOOK_URL` (Render → Settings → Deploy Hook) and `VERCEL_DEPLOY_HOOK_URL` (Vercel → Settings → Git → Deploy Hooks, branch `main`); variables `API_URL` = `https://YOUR-API.onrender.com` and `WEB_ORIGIN` = `https://YOUR-APP.vercel.app`. Optionally add yourself as **Required reviewer**.
4. Turn automatic deploys **off**: Render → Settings → Auto-Deploy **No**; for Vercel add `"git": { "deploymentEnabled": false }` to `apps/web/vercel.json`.
5. To release: **Actions** → **Deploy** → **Run workflow** → **production**.
6. After the first run, confirm Vercel → Deployments shows a deployment made by the hook. If not, remove the `git` line again and keep Vercel on automatic deploys.

NOTE

The free Supabase plan limits how many projects you can have. The test project may need a second Supabase organisation or a paid plan.

6. Rollback and backups

6.1 Undo a bad release

1. Website: **Vercel** → **Deployments**, open the previous good production deployment → **Instant Rollback** (or ... → **Promote to Production**).
2. API: **Render** → **a-ai-api** → **Events**, find the previous good deploy → **Rollback**. The code goes back; the database stays as it is (safe, because migrations only add things).
3. Rollback may turn automatic deploys off. Turn them on again after you have fixed and merged the problem.
4. Run `pnpm smoke --api https://YOUR-APP.vercel.app --web https://YOUR-APP.vercel.app`.
5. Or simply revert the bad commit on GitHub (open the merged pull request → **Revert**) and merge the revert; both services redeploy.

6.2 Backups

Data	Backup	Restore
PostgreSQL (accounts, chats, history)	Supabase backups for your plan (check Database → Backups)	Supabase → Database → Backups. On free plans consider a manual export (<code>pg_dump</code> with the session pooler URL) now and then.
Redis (sessions, quotas)	Not needed	Nothing to restore; guests get new sessions.
Uploaded images	Not included in database backups	Lost files show as "Unavailable".

7. Troubleshooting

Always look at **Render** → **a-ai-api** → **Logs** first. At start-up the API lists wrong variable names (never their values).

What you see	Cause and fix
Signed in, but immediately signed out; guest chat fails; history empty	The browser is calling onrender.com directly. Delete <code>VITE_API_URL</code> in Vercel, make sure the rewrites exist (Step 17), redeploy.
Error "This request came from a site that is not allowed to make changes"	<code>CORS_ORIGIN</code> on Render does not exactly match <code>https://YOUR-APP.vercel.app</code> (check <code>https://</code> , spelling, no trailing slash). Use the stable production address, not a preview URL.
<code>/health</code> on the Vercel address shows the website instead of JSON	Rewrites missing or placed after the <code>/(.*)</code> rule. Fix <code>vercel.json</code> order and redeploy.
<code>ROUTER_EXTERNAL_TARGET_ERROR</code> or 504 from Vercel	Render was asleep or busy for more than 120 seconds. Retry after a minute; consider the keep-awake ping (Step 27).
Very long answers stop in the middle	Proxy timeout. Keep answers shorter, or move to a custom domain (the main deployment guide), which avoids the proxy.
Image upload fails only for big files	Try a smaller image. If large uploads always fail through Vercel, set <code>ATTACHMENT_MAX_BYTES=4000000</code> on Render.
<code>REDIS_URL: production requires TLS (rediss://)</code>	Use the Upstash <code>rediss://</code> URL.
<code>CORS_ORIGIN.0: production origins must use https://</code>	Set <code>https://YOUR-APP.vercel.app</code> with <code>https://</code> .
<code>RESEND_API_KEY: production requires a real email provider</code>	Add the Resend key (Step 7).
<code>JWT_SECRET: must be at least 32 characters</code>	Generate one (Step 9).
set <code>SUPABASE_URL</code> and <code>SUPABASE_SERVICE_ROLE_KEY</code> together	Set both, or remove both (uploads switch off).
Build fails at <code>prisma migrate deploy</code> (P1001, cannot reach database)	<code>DIRECT_URL</code> must be the Session pooler string (port 5432, host <code>pooler.supabase.com</code>), with the right password. Also check the Supabase project is not paused.
Build fails: <code>tsc</code> or <code>turbo</code> not found	The build command must start with <code>corepack enable && NODE_ENV=development pnpm install ...</code> (Step 11).
<code>/ready</code> returns 503	The JSON names what is down: database (paused Supabase? wrong URL?), Redis, or no AI key.
Everyone hits "too many requests" together	<code>TRUST_PROXY_HOPS</code> must be <code>2</code> in this setup.
Verification email never arrives	Test mode only sends to your own Resend email. Verify a domain (Step 7, option B). Check Resend → Logs and spam.
Email link opens the wrong site	<code>APP_URL</code> on Render must be <code>https://YOUR-APP.vercel.app</code> .
First request takes about a minute	Normal on Render free (the service was asleep). Step 27 keeps it awake.
Microphone button missing	<code>GROQ_API_KEY</code> not set.
Cleanup workflow fails	Check the three Actions secrets (Step 23) and run it again from the Actions tab.

Every API error carries a `requestId` (also the `x-request-id` response header). Search for it in Render → Logs.

Appendix A: your final vercel.json

For reference, the complete `apps/web/vercel.json` after Step 17 (security headers unchanged from the repository):

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "framework": "vite",
  "installCommand": "cd ../../ && pnpm install --frozen-lockfile",
  "buildCommand": "cd ../../ && pnpm turbo run build --filter=@a-ai/web...",
  "outputDirectory": "dist",
  "rewrites": [
    { "source": "/api/:path*", "destination": "https://YOUR-API.onrender.com/api/:path*" },
    { "source": "/health", "destination": "https://YOUR-API.onrender.com/health" },
    { "source": "/ready", "destination": "https://YOUR-API.onrender.com/ready" },
    { "source": "/(.*)", "destination": "/index.html" }
  ],
  "headers": [
    { "source": "/(.*)", "headers": [ ...unchanged security headers... ] },
    {
      "source": "/api/(.*)",
      "headers": [{ "key": "x-vercel-enable-rewrite-caching", "value": "0" }]
    },
    {
      "source": "/assets/(.*)",
      "headers": [{ "key": "Cache-Control", "value": "public, max-age=31536000, immutable" }]
    }
  ]
}
```

Appendix B: settings summary

Render (a-ai-api, Free)

Field	Value
Root Directory	(empty)
Build Command	<code>corepack enable && NODE_ENV=development pnpm install --frozen-lockfile && pnpm turbo run build --filter=@a-ai/api... && pnpm db:deploy</code>
Start Command	<code>node apps/api/dist/server.js</code>
Health Check Path	<code>/health</code>
Auto-Deploy	On Commit (or No with the Deploy workflow)
Environment	<code>NODE_VERSION, NODE_ENV, HOST, LOG_LEVEL, TRUST_PROXY_HOPS=2, DATABASE_URL, DIRECT_URL, REDIS_URL, JWT_SECRET, CORS_ORIGIN, APP_URL, RESEND_API_KEY, EMAIL_FROM, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_STORAGE_BUCKET, provider keys</code>

Optional API limits (defaults): `GUEST_DAILY_MESSAGE_LIMIT=20, USER_DAILY_MESSAGE_LIMIT=200, GUEST_COMPARE_MAX_MODELS=2, USER_COMPARE_MAX_MODELS=4, GUEST_SESSION_TTL_MINUTES=1440, ATTACHMENT_MAX_BYTES=5242880, AUDIO_MAX_BYTES=10485760, CONTEXT_SUMMARY_ENABLED=true, CHAT_FALLBACK_ENABLED=true.`

Vercel (website)

Field	Value
Root Directory	<code>apps/web</code>
Framework / commands	From <code>apps/web/vercel.json</code>
Production Branch	<code>main</code>
<code>VITE_API_URL</code>	not set (empty)
<code>VITE_SENTRY_DSN</code>	Optional

GitHub

Item	Value
Branch ruleset on <code>main</code>	Pull request + CI checks required
Actions secrets	<code>DATABASE_URL, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY</code>
Workflows	<code>ci.yml</code> (automatic), <code>cleanup.yml</code> (daily), <code>deploy.yml</code> (optional, Section 5)

Appendix C: checklist

- [] `pnpm verify` passes; latest code is on `main`; CI green
- [] Supabase: pooler strings copied (6543 and 5432), private bucket `a-ai-attachments`, `service_role` key
- [] Upstash `rediss://` URL
- [] Resend key; email option chosen (test mode or verified domain)
- [] At least one AI key; `JWT_SECRET` generated
- [] Render: build command ends with `pnpm db:deploy`; start command; `/health` check; all variables incl. `TRUST_PROXY_HOPS=2`, `CORS_ORIGIN` and `APP_URL` = your vercel.app address
- [] Render `/health` and `/ready` OK
- [] Vercel: Root Directory `apps/web`; `VITE_API_URL` removed; rewrites added and deployed
- [] `YOUR-APP.vercel.app/health` returns the API JSON
- [] Browser Network tab shows `/api` calls on vercel.app; `__Host-` cookies present
- [] Smoke test passes; manual sign-up, chat, compare, history work
- [] `main` protected; cleanup workflow added and run once
- [] Admin promoted; optional keep-awake ping set up